Strict-Additive Fees Reinvested Inside Pricing for AMMs

Conservative and Dissipative Constructions with ExactIn/ExactOut

Vadim Fadeev
xboxfadeev@gmail.com

Sergey Prilutskiy
prilutski@gmail.com

Vijay Singh
vijaykumar.singh@openzeppelin.com

Sergej Kunz
info@deacix.de

February 21, 2026

Acknowledgements. The authors thank Nikita Galikhanov (n.galikhanov@1inch.io), Gleb Alekseev (g.alekseev@1inch.io), and Vasiliy Tikhonenko (v.tikhonenko@1inch.io) for valuable discussions and feedback.

Abstract

Constant-product AMMs ($xy = k$) are strictly additive (split-invariant): executing a trade of size $a + b$ yields the same final pool state as executing a and then b. This paper studies fee mechanisms where fees are *reinvested inside pricing* (no external fee buckets) while preserving strict additivity. We show that strict additivity forces a telescoping structure that can be expressed as an invariant of the form $K = y\,\Psi(x)$. This yields a *conservative* design that is strictly additive for ExactIn and ExactOut and is invertible (a perfect round trip returns the pool to the start when the trader swaps back exactly what they received). We then introduce a *dissipative* design: a single deterministic mapping where the power α is always applied to the input token's reserve. This mapping is strictly additive for splits in both directions, but is non-conservative—round trips cost the trader and accumulate value in reserves, creating a real bid-ask spread for economic incentive. We prove an *impossibility theorem*: no stateless fee mechanism can simultaneously be progressive (larger trades pay higher percentage) and LP-protective (splitting cannot reduce fees). The dissipative design achieves statelessness and LP-protection at the cost of being anti-progressive; we quantify this trade-off with economic analysis and numerical evidence, and discuss alternative designs including cumulative volume fees. We provide full intermediate derivations and numeric examples.

Contents

1 Setup **2**

2 Baseline: constant product $xy = k$ **3**

3 Why the cocycle condition appears **3**

4 From R to Ψ: a better description of the potential function **4**

4.1 Telescoping solution via an endpoint potential G 4

4.2 Defining Ψ and its economic meaning 4

5 Conservative Design (Single Invariant) **5**
5.1 ExactIn $X \to Y$ . . . 5
5.2 ExactOut $X \to Y$: full derivation . . . 5
5.3 Strict additivity checks . . . 5
5.4 Why conservative design allows perfect round trips . . . 5
5.5 Reinvested fee accounting relative to constant product . . . 6

6 Conservative power family: $\Psi(x) = x^\alpha$ **6**

7 Dissipative Design (Single Deterministic Mapping) **6**
7.1 The unified dissipative mapping . . . 7
7.2 Why "dissipative" is the right framing . . . 7
7.3 Symmetric power example: $\Psi(t) = t^\alpha$ . . . 7

8 Why Strict Additivity Matters **8**
8.1 Formal additivity types for trader output . . . 8
8.2 How standard fee rules break strict additivity on $xy = k$ . . . 8
8.3 Why path-independence is the right design goal . . . 8

9 The Impossibility Theorem **9**
9.1 Formal definitions . . . 9
9.2 Key lemma: progressive implies superadditivity at fixed state . . . 10
9.3 Main theorem . . . 10
9.4 Geometric interpretation . . . 11
9.5 Information-theoretic intuition . . . 11
9.6 Can changing the curve escape the impossibility? . . . 11

10 Economic Analysis of the Dissipative Design **12**
10.1 Effective fee rate . . . 12
10.2 Fee rate by trade size ($\alpha = 0.997$) . . . 13
10.3 Comparison with alternative fee mechanisms . . . 13
10.4 Alternative: cumulative volume design . . . 13
10.5 Why stateless is preferred for the dissipative design . . . 13

11 Numeric showcase and evidence **14**
11.1 Fee rate vs trade size ($\alpha = 0.997$) . . . 14
11.2 Round-trip loss summary ($\alpha = 0.997$) . . . 14
11.3 Detailed hand-checkable examples . . . 15

12 Pitfall: scaling ExactOut input by a constant breaks strict additivity **16**

13 Summary **16**
13.1 Mechanism comparison . . . 17
13.2 Decision table . . . 17

1 Setup

Definition 1.1 (State)**.** The pool state is reserves $(x, y) \in \mathbb{R}^2_{>0}$, where x is token X reserve and y is token Y reserve.

Definition 1.2 (State update map)**.** Let $F_\theta : \mathbb{R}^2_{>0} \to \mathbb{R}^2_{>0}$ be a state update map parameterized by a trade parameter θ. For ExactIn we take $\theta = \Delta x$; for ExactOut we take $\theta = \Delta y$. We write

$$F_\theta(x, y) = (x_\theta, y_\theta).$$

Definition 1.3 (Strict additivity / split invariance)**.** A family $\{F_\theta\}$ is *strictly additive* in parameter θ if for all $a, b > 0$,

$$F_{a+b} = F_b \circ F_a.$$

Equivalently, applying a then b produces the same final state as applying $a + b$ once.

Definition 1.4 (Output function)**.** For ExactIn (parameter Δx), define

$$\Delta y = f((x, y), \Delta x) := y - \pi_y(F_{\Delta x}(x, y)),$$

where $\pi_y(x', y') = y'$.

2 Baseline: constant product $xy = k$

Lemma 2.1 (Constant product ExactIn)**.** *With invariant $xy = k$ and full input credit $x' = x+\Delta x$, the post-swap y is uniquely*

$$y' = \frac{xy}{x + \Delta x} = y\frac{x}{x + \Delta x},$$

and

$$\Delta y_{\text{cp}} = y - y' = y\frac{\Delta x}{x + \Delta x}.$$

Lemma 2.2 (Strict additivity for constant product)**.** *Constant product is strictly additive for ExactIn.*

Proof. Let $\Delta x = a+b$. After a, $y_1 = xy/(x+a)$. After b, $y_2 = (x+a)y_1/(x+a+b) = xy/(x+a+b)$, equal to the one-shot result. □

3 Why the cocycle condition appears

A common way to model reinvested fees is to start from constant product and multiply by a factor $R(x, \Delta) \geq 1$ that keeps extra Y in the pool:

$$y'(x, \Delta) = y\frac{x}{x + \Delta}R(x, \Delta).$$

Theorem 3.1 (Strict additivity implies the cocycle identity)**.** *The update rule*

$$F_\Delta(x, y) = \left(x + \Delta,\ y\frac{x}{x + \Delta}R(x, \Delta)\right)$$

is strictly additive in Δ iff

$$\boxed{R(x, a)R(x + a, b) = R(x, a + b)} \qquad \forall x, a, b > 0.$$

Proof. Split $\Delta = a + b$. After a:

$$y_1 = y\frac{x}{x + a}R(x, a), \quad x_1 = x + a.$$

After b:

$$y_2 = y_1\frac{x_1}{x_1 + b}R(x_1, b) = \left(y\frac{x}{x + a}R(x, a)\right)\frac{x + a}{x + a + b}R(x + a, b).$$

Cancel $x + a$:

$$y_2 = y\frac{x}{x + a + b}R(x, a)R(x + a, b).$$

One-shot:

$$y_S = y\frac{x}{x + a + b}R(x, a + b).$$

Strict additivity requires $y_2 = y_S$ for all $y > 0$, hence the boxed identity. □

4 From R to Ψ: a better description of the potential function

4.1 Telescoping solution via an endpoint potential G

Theorem 4.1 (General telescoping form)**.** *If the cocycle identity holds, then (under mild regularity) there exists $G : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$ such that*

$$\boxed{R(x, \Delta) = \frac{G(x)}{G(x + \Delta)}}.$$

Conversely, any such ratio satisfies the cocycle identity.

Proof. Sufficiency is immediate:

$$\frac{G(x)}{G(x + a)} \cdot \frac{G(x + a)}{G(x + a + b)} = \frac{G(x)}{G(x + a + b)}.$$

Necessity can be shown by fixing a reference point and defining G by endpoint products so that intermediate factors cancel. □

4.2 Defining Ψ and its economic meaning

Substitute $R(x, \Delta) = G(x)/G(x + \Delta)$:

$$y' = y\frac{x}{x + \Delta}\frac{G(x)}{G(x + \Delta)} = y\frac{xG(x)}{(x + \Delta)G(x + \Delta)}.$$

Define the combined potential coordinate

$$\boxed{\Psi(x) := xG(x)}.$$

Then the update becomes

$$\boxed{x' = x + \Delta, \qquad y' = y\frac{\Psi(x)}{\Psi(x + \Delta)}.} \tag{1}$$

Proposition 4.2 (Invariant)**.** *The quantity*

$$\boxed{K := y\Psi(x)}$$

is invariant under the update (1)*.*

Proof. Multiply both sides of (1) by $\Psi(x') = \Psi(x + \Delta)$:

$$y'\Psi(x') = y\frac{\Psi(x)}{\Psi(x')}\Psi(x') = y\Psi(x).$$

□

Remark 4.3 (Economic interpretation of Ψ)*.* Ψ is a monotone (typically increasing) re-parameterization of the x-reserve. The swap outcome depends only on the endpoint ratio $\Psi(x)/\Psi(x')$. You can interpret $\Psi(x)$ as the pool's *effective* or *virtual* X-liquidity coordinate. A different Ψ changes the curvature of the AMM and therefore changes how much output is paid for the same input.

Proposition 4.4 (Marginal price)**.** *On the invariant curve $y\Psi(x) = K$, the instantaneous price (marginal Y out per X in) is*

$$\boxed{p(x, y) = -\frac{dy}{dx} = \frac{y\Psi'(x)}{\Psi(x)}.}$$

Proof. Differentiate $y\Psi(x) = K$: $\Psi(x)dy + y\Psi'(x)dx = 0$. Rearranging yields the formula. □

5 Conservative Design (Single Invariant)

The conservative design uses one function Ψ for both directions, i.e., one conserved quantity $K = y\Psi(x)$. It is strictly additive and invertible (time-reversible).

5.1 ExactIn $X \to Y$

From (1), the output is

$$\boxed{\Delta y = y - y' = y\left(1 - \frac{\Psi(x)}{\Psi(x + \Delta x)}\right).} \tag{2}$$

5.2 ExactOut $X \to Y$: full derivation

Given a target $0 < \Delta y < y$, set $y' = y - \Delta y$. Conservation of $K = y\Psi(x)$ implies

$$(y - \Delta y)\Psi(x') = y\Psi(x).$$

Solve for $\Psi(x')$:

$$\Psi(x') = \Psi(x)\frac{y}{y - \Delta y}.$$

Assuming Ψ is invertible:

$$x' = \Psi^{-1}\left(\Psi(x)\frac{y}{y - \Delta y}\right), \qquad \boxed{\Delta x = x' - x = \Psi^{-1}\left(\Psi(x)\frac{y}{y - \Delta y}\right) - x.}$$

5.3 Strict additivity checks

Lemma 5.1 (ExactIn strict additivity for Curve A)**.** *For all $a, b > 0$,*

$$F_{a+b}^{A,\text{in}} = F_b^{A,\text{in}} \circ F_a^{A,\text{in}}.$$

Proof. After a: $y_1 = y\Psi(x)/\Psi(x + a)$. After b: $y_2 = y_1\Psi(x + a)/\Psi(x + a + b) = y\Psi(x)/\Psi(x + a + b)$. □

Lemma 5.2 (ExactOut strict additivity for Curve A)**.** *For $a, b > 0$ with $a + b < y$, the ExactOut map is strictly additive in Δy.*

Proof. Write $r(\Delta y) = y/(y - \Delta y)$. Note that $r(a)\, r(b \mid y - a) = y/(y - a) \cdot (y - a)/(y - a - b) = y/(y - a - b) = r(a + b)$. Since ExactOut is $\Psi(x') = \Psi(x)r(\Delta y)$, endpoint multiplication yields the same $\Psi(x')$ and hence the same x'. □

5.4 Why conservative design allows perfect round trips

Proposition 5.3 (Round trip is an identity (ideal real arithmetic))**.** *In the conservative design, perform ExactIn $X \to Y$ with input Δx producing output Δy. Then input exactly that Δy back in the reverse direction. The pool returns to the initial state (modulo rounding).*

Proof. Both legs preserve the same invariant $K = y\Psi(x)$. After the first leg, the trader received $\Delta y = y - y'$. Inputting Δy back restores y to its initial value. With y restored and K unchanged, $\Psi(x)$ must also be restored, implying x returns (for invertible Ψ). □

5.5 Reinvested fee accounting relative to constant product

For the same Δx, constant product yields $y_{\text{cp}} = y\,x/(x+\Delta x)$. The conservative design yields $y' = y\Psi(x)/\Psi(x+\Delta x)$. Define retained Y vs constant product:

$$\boxed{\text{fee}_Y^{(\text{vs CP})} = y' - y_{\text{cp}} = y\left(\frac{\Psi(x)}{\Psi(x+\Delta x)} - \frac{x}{x+\Delta x}\right).}$$

This quantity measures how much more Y remains in the pool compared to pure constant product for the same input.

6 Conservative power family: $\Psi(x) = x^\alpha$

Let $0 < \alpha \le 1$ and $\Psi(x) = x^\alpha$. Then invariant is

$$\boxed{K = x^\alpha y.}$$

ExactIn output is

$$\boxed{\Delta y = y\left(1 - \left(\frac{x}{x+\Delta x}\right)^\alpha\right).}$$

ExactOut input for target Δy is

$$\boxed{\Delta x = x\left(\left(\frac{y}{y-\Delta y}\right)^{1/\alpha} - 1\right).}$$

Marginal price becomes

$$\boxed{p = \alpha\frac{y}{x}.}$$

Implementation formulas (ExactIn / ExactOut). For on-chain implementations it is useful to expose closed forms for both swap directions. We write the trader input as Δx_{in} or Δy_{in} and the trader output as Δy_{out} or Δx_{out}.

(i) $X \to Y$ direction.

$$\textbf{ExactIn:}\quad \Delta y_{\text{out}} = y - y' = y\left(1 - \left(\frac{x}{x+\Delta x_{\text{in}}}\right)^\alpha\right), \tag{3}$$

$$\textbf{ExactOut:}\quad \Delta x_{\text{in}} = x\left(\left(\frac{y}{y-\Delta y_{\text{out}}}\right)^{1/\alpha} - 1\right), \qquad 0 < \Delta y_{\text{out}} < y. \tag{4}$$

(ii) $Y \to X$ direction.

$$\textbf{ExactIn:}\quad \Delta x_{\text{out}} = x - x' = x\left(1 - \left(\frac{y}{y+\Delta y_{\text{in}}}\right)^\alpha\right), \tag{5}$$

$$\textbf{ExactOut:}\quad \Delta y_{\text{in}} = y\left(\left(\frac{x}{x-\Delta x_{\text{out}}}\right)^{1/\alpha} - 1\right), \qquad 0 < \Delta x_{\text{out}} < x. \tag{6}$$

7 Dissipative Design (Single Deterministic Mapping)

The dissipative design uses a *single* mapping rule: the power α is always applied to the **input token's reserve**. This is not two separate curves, but one unified, asymmetric mapping that is non-conservative (dissipative).

7.1 The unified dissipative mapping

The key insight is that there is **one rule**: when token T is input, apply $\Psi_T(t) = t^\alpha$ to its reserve. This yields a single parameterized family of state updates:

$$\boxed{F_{\Delta_{\text{in}}}(x, y) = \begin{cases} \left(x + \Delta x,\ y\left(\dfrac{x}{x + \Delta x}\right)^\alpha\right) & \text{when } X \text{ is input,} \\ \left(x\left(\dfrac{y}{y + \Delta y}\right)^\alpha,\ y + \Delta y\right) & \text{when } Y \text{ is input.} \end{cases}}$$

Each direction is strictly additive in its own input parameter by telescoping.

7.2 Why "dissipative" is the right framing

- **Single rule**: Power the input token's reserve ratio—one formula, not two curves.
- **Deterministic**: Given (state, input, direction), output is uniquely determined.
- **Non-conservative**: Round trips lose value (like friction in physics).
- **Time-irreversible**: Unlike conservative Hamiltonian systems, you cannot reverse time to recover the initial state.

7.3 Symmetric power example: $\Psi(t) = t^\alpha$

With $0 < \alpha < 1$:

$$X \to Y: \quad y' = y\left(\frac{x}{x + \Delta x}\right)^\alpha.$$

$$Y \to X: \quad x' = x\left(\frac{y}{y + \Delta y}\right)^\alpha.$$

Unlike the conservative design, these directions do not preserve a single global K, so round trips dissipate trader value into the pool.

Implementation formulas (ExactIn / ExactOut). Even though this section defines a *dissipative* mapping (no invariant), the per-swap closed forms are identical to Section 6, because each direction still follows the same one-step update rule.

(i) $X \to Y$ direction.

$$\textbf{ExactIn:} \quad \Delta y_{\text{out}} = y - y' = y\left(1 - \left(\frac{x}{x + \Delta x_{\text{in}}}\right)^\alpha\right), \tag{7}$$

$$\textbf{ExactOut:} \quad \Delta x_{\text{in}} = x\left(\left(\frac{y}{y - \Delta y_{\text{out}}}\right)^{1/\alpha} - 1\right), \qquad 0 < \Delta y_{\text{out}} < y. \tag{8}$$

(ii) $Y \to X$ direction.

$$\textbf{ExactIn:} \quad \Delta x_{\text{out}} = x - x' = x\left(1 - \left(\frac{y}{y + \Delta y_{\text{in}}}\right)^\alpha\right), \tag{9}$$

$$\textbf{ExactOut:} \quad \Delta y_{\text{in}} = y\left(\left(\frac{x}{x - \Delta x_{\text{out}}}\right)^{1/\alpha} - 1\right), \qquad 0 < \Delta x_{\text{out}} < x. \tag{10}$$

8 Why Strict Additivity Matters

A natural question is why strict additivity (split-invariance) should be a design goal at all. After all, the standard Curve-style output-fee rule is *superadditive*: splitting a trade yields strictly *more* output than one-shot execution, which is "good for traders." However, superadditivity still creates path dependence, which has concrete negative consequences.

8.1 Formal additivity types for trader output

Definition 8.1 (Additivity types)**.** Let $\mathrm{Out}(\Delta; x, y) := y - \pi_y(F_\Delta(x, y))$ be the trader output and

$$\mathrm{Out}_{\mathrm{split}}(a, b; x, y) := \mathrm{Out}(a; x, y) + \mathrm{Out}(b;\, F_a(x, y))$$

the total output from executing a then b. The mechanism is:

(a) **Strictly additive** if $\mathrm{Out}(a{+}b) = \mathrm{Out}_{\mathrm{split}}(a, b)$ for all $a, b \geq 0$. (Path-independent.)

(b) **Subadditive** if $\mathrm{Out}(a{+}b) > \mathrm{Out}_{\mathrm{split}}(a, b)$ for all $a, b > 0$. (One-shot is better; splitting hurts the trader.)

(c) **Superadditive** if $\mathrm{Out}(a{+}b) < \mathrm{Out}_{\mathrm{split}}(a, b)$ for all $a, b > 0$. (Split is better; one-shot hurts the trader.)

8.2 How standard fee rules break strict additivity on $xy = k$

Both common fee mechanisms applied to constant product create path dependence:

Output-fee (Curve-style) is superadditive. Start from (x, y) with $k = xy$. The fee-free output for input Δ is $\Delta y_{\mathrm{nf}} = y\Delta/(x{+}\Delta)$. The rule: pay trader $(1{-}f)\Delta y_{\mathrm{nf}}$ and keep $f\Delta y_{\mathrm{nf}}$ in Y. After the trade, $y_1 = xy/(x{+}\Delta) + fy\Delta/(x{+}\Delta) = y(x{+}f\Delta)/(x{+}\Delta)$, so

$$k_1 = (x{+}\Delta)\, y_1 = y(x + f\Delta) > xy = k_0.$$

The fee pushes k upward. The second chunk of a split trade executes on a "richer" pool (higher k), yielding more output than the corresponding portion of a one-shot trade. Splitting is strictly better for the trader: *superadditive.*

Input-fee reinvest is subadditive. Trade only the net amount $(1{-}f)\Delta$ on the constant-k curve: $y_{\mathrm{nf}} = xy/(x{+}(1{-}f)\Delta)$. Credit the full Δ to reserves: $x_1 = x + \Delta$. Now $k_1 = x_1\, y_{\mathrm{nf}} = (x{+}\Delta) \cdot xy/(x{+}(1{-}f)\Delta) > xy$ since $x{+}\Delta > x{+}(1{-}f)\Delta$. The extra X makes the pool more X-heavy; subsequent $X \to Y$ trades face worse terms. Splitting is strictly worse for the trader: *subadditive.*

Dissipative r^α is strictly additive. By telescoping (Section 4), the outcome depends only on the ratio $\Psi(x)/\Psi(x{+}\Delta)$. Intermediate terms cancel identically regardless of how Δ is split. There is no path dependence: *strictly additive.*

8.3 Why path-independence is the right design goal

(a) **Aggregator/router neutrality.** If the swap outcome depends on how execution is split, routing algorithms must simulate all possible split patterns to find the optimal one. Strict additivity means routers can split arbitrarily without affecting the final outcome—the optimization problem collapses.

(b) **Partial fills and TWAP.** Time-weighted execution, partial limit-order fills, and MEV-resistant chunking all require that outcome does not depend on chunking granularity. With superadditive fees, a TWAP order gets a different outcome depending on chunk sizes—an undesirable property.

(c) **Deterministic pool growth.** Under strict additivity, the pool's effective liquidity grows deterministically with volume, independent of how trades are split. This makes LP returns predictable from aggregate volume alone.

(d) **Composability.** Protocols building on top of the AMM (vaults, strategies, aggregators) can reason about outcomes without simulating all possible split patterns.

(e) **No gaming incentive.** With superadditive fees, sophisticated traders split trades to gain advantage. With subadditive fees, they batch. Both create unfair advantages. Strict additivity eliminates the gaming dimension entirely.

Remark 8.2 (SwapVM compatibility). In the SwapVM architecture, orders are bytecode programs executed by an on-chain VM. Each program specifies its pricing instruction (e.g., dissipative power-curve swap). Strict additivity ensures that the program's economic outcome is independent of how the taker chooses to fill the order—critical for a system where makers publish programs and takers execute them.

Fee Rule	**Additivity**	**For Trader**	**Path-Dep.?**	k **Change**
Fee-free $xy = k$	Strictly additive	Path-independent	No	k constant
Output-fee (Curve)	Superadditive	Split better	Yes	k increases (in Y)
Input-fee reinvest	Subadditive	One-shot better	Yes	k increases (in X)
Dissipative r^α	Strictly additive	Path-independent	No	$K = yx^\alpha$ constant

9 The Impossibility Theorem

We now prove that the dissipative design's anti-progressive fee structure is not a design flaw but a *mathematical necessity*: no stateless fee mechanism can simultaneously be progressive and LP-protective.

9.1 Formal definitions

Definition 9.1 (Fee function)**.** A *fee function* is a mapping $f : \mathbb{R}_{>0} \times \mathcal{S} \to \mathbb{R}_{\geq 0}$ where $\Delta \in \mathbb{R}_{>0}$ is the trade size, $s \in \mathcal{S}$ is the system state, and $f(\Delta, s)$ is the fee charged.

Definition 9.2 (Stateless)**.** A fee function is *stateless* if s consists only of the current reserves (x, y): $f(\Delta, s) = f(\Delta, x, y)$. No historical information is required.

Definition 9.3 (Progressive)**.** A fee function is *progressive* if the fee rate $\varphi(\Delta, s) := f(\Delta, s)/\Delta$ is strictly increasing in Δ at fixed s:

$$\Delta_2 > \Delta_1 \implies \frac{f(\Delta_2, s)}{\Delta_2} > \frac{f(\Delta_1, s)}{\Delta_1}.$$

Definition 9.4 (LP-Protective)**.** A fee function is *LP-protective* if splitting a trade cannot reduce the total fee:

$$f(a + b,\ s) \ \leq\ f(a,\ s) + f(b,\ s_a)$$

where s_a is the state after executing trade a.

9.2 Key lemma: progressive implies superadditivity at fixed state

Lemma 9.5. *If a fee function f is progressive, then at any fixed state s:*

$$f(a+b,\, s) > f(a,\, s) + f(b,\, s) \qquad \forall\, a, b > 0.$$

Proof. Let $\varphi(\Delta) := f(\Delta, s)/\Delta$. Progressive means φ is strictly increasing, so $\varphi(a+b) > \varphi(a)$ and $\varphi(a+b) > \varphi(b)$. Then:

$$f(a+b) = a\,\varphi(a+b) + b\,\varphi(a+b) > a\,\varphi(a) + b\,\varphi(b) = f(a) + f(b). \qquad \square$$

9.3 Main theorem

Theorem 9.6 (Impossibility of Stateless + Progressive + LP-Protective)**.** *There exists no fee function that is simultaneously stateless, progressive, and LP-protective.*

Proof. Suppose for contradiction that such a function f exists. Fix a state s_0 and a total trade size $D > 0$. Split D into n equal pieces of size D/n and let s_i denote the state after the i-th sub-trade.

Step 1 (LP-protective gives an upper bound). Applying the LP-protective inequality $n-1$ times:

$$f(D,\, s_0) \;\le\; \sum_{i=0}^{n-1} f\left(\frac{D}{n},\, s_i\right). \tag{11}$$

Step 2 (State changes are second-order for small sub-trades). Since f is stateless, s_i depends only on current reserves. After i sub-trades of size D/n, the reserves have shifted by a total of iD/n. For a smooth fee function, Taylor expansion gives:

$$f\left(\frac{D}{n},\, s_i\right) = f\left(\frac{D}{n},\, s_0\right) + O\left(\frac{D^2}{n^2}\right).$$

Summing over n terms:

$$\sum_{i=0}^{n-1} f\left(\frac{D}{n},\, s_i\right) = n \cdot f\left(\frac{D}{n},\, s_0\right) + O\left(\frac{D^2}{n}\right) = D \cdot \varphi\left(\frac{D}{n},\, s_0\right) + O\left(\frac{D^2}{n}\right).$$

Step 3 (Progressive rate reduction is first-order). Since φ is strictly increasing in Δ and $D/n < D$ for $n \ge 2$:

$$\varphi\left(\frac{D}{n}, s_0\right) < \varphi(D, s_0).$$

Taking $n \to \infty$:

$$\lim_{n\to\infty}\left[D \cdot \varphi\left(\frac{D}{n}, s_0\right) + O\left(\frac{D^2}{n}\right)\right] = D \cdot \varphi(0^+, s_0) < D \cdot \varphi(D, s_0) = f(D, s_0).$$

Step 4 (Contradiction). For sufficiently large n, the right side of (11) is strictly less than $f(D, s_0)$, contradicting (11). $\square$

Remark 9.7 (Why this proof handles state changes correctly). Unlike the naïve argument that reduces LP-protection to same-state subadditivity $f(a+b) \le f(a) + f(b)$ (which is valid only when state doesn't change between sub-trades), this proof accounts for state evolution. The key insight is that for infinitesimal splits, state changes are *second-order* ($O(D^2/n^2)$ per step, $O(D^2/n)$ total), while the progressive rate reduction from $\varphi(D)$ to $\varphi(D/n)$ is *first-order*.

Corollary 9.8 (Design space constraint)**.** *Any practical stateless fee mechanism must sacrifice either progressivity or LP-protection:*

Design	***Stateless***	***Progressive***	***LP-Protective***	***Sacrifice***
Flat fee	✓	—	✓	*Progressive*
Dissipative r^α	✓	× *(anti)*	✓	*Progressive*
Per-trade progressive	✓	✓	×	*LP-Protective*
Cumulative volume	×	✓	✓	*Stateless*

The dissipative design occupies the "stateless + LP-protective" cell, necessarily sacrificing progressivity.

9.4 Geometric interpretation

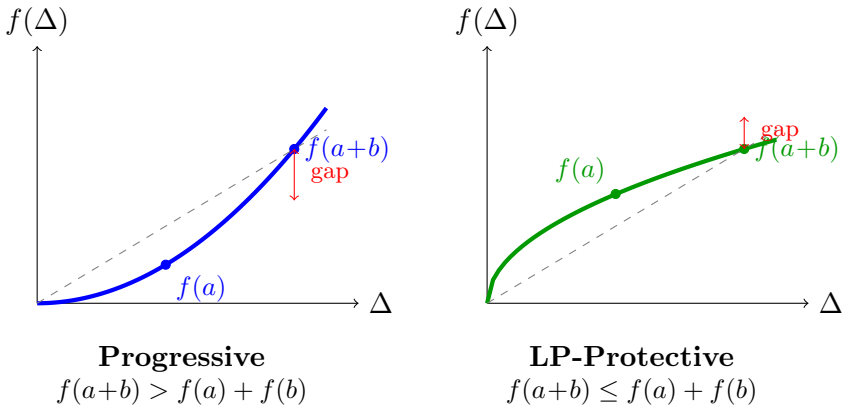

Figure 1: Progressive fees are convex (superadditive at fixed state), while LP-protective fees are concave (subadditive). No function can be both.

9.5 Information-theoretic intuition

The impossibility has a clean information-theoretic interpretation:

1. **Progressive** requires "knowing" a trade is large → charge more.
2. **LP-Protective** means the mechanism cannot distinguish a single large trade from many small trades (split-invariant).
3. **Stateless** means no memory of previous trades.

If the mechanism cannot distinguish large from split-small (LP-protective) and has no memory (stateless), it *cannot know* a trade is large (progressive). Changing the AMM curve does not create information that doesn't exist—the constraint is fundamental, not formula-specific.

9.6 Can changing the curve escape the impossibility?

As proven in Sections 3–4, *any* monotone $\Psi : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$ yields a strictly additive (telescoping) invariant $K = y\Psi(x)$. Telescoping gives LP-protection for free. The question is whether some non-power Ψ can achieve progressive implicit fees.

Theorem 9.9 (Anti-progressive structure for general Ψ)**.** *For any potential function Ψ with positive implicit fees (i.e., the trader receives less than constant product), the fee structure is anti-progressive.*

Proof. The curve gives output $\Delta y = y(1 - \Psi(x)/\Psi(x + \Delta))$. When a trade of size Δ executes, the x-reserve moves from x to $x + \Delta$. The *marginal implicit fee rate* at reserve level x is the infinitesimal fee per unit input:

$$\mu(x) := \lim_{\delta \to 0} \frac{\text{fee}(\delta;\, x)}{y\,\delta} = \frac{1}{x} - \frac{\Psi'(x)}{\Psi(x)}.$$

(This follows from L'Hôpital on the difference between constant-product and Ψ-curve outputs.) Positive fees require $\mu(x) > 0$, i.e., $\Psi'(x)/\Psi(x) < 1/x$, meaning Ψ grows slower than linear.

For the overall fee to be *progressive* in trade size, $\mu(x)$ must be increasing in x (so that later units of a large trade pay more than earlier units). Differentiating:

$$\mu'(x) = -\frac{1}{x^2} - \frac{\Psi''\Psi - (\Psi')^2}{\Psi^2}.$$

Substituting $\Psi = e^g$ (so $\Psi'/\Psi = g'$ and $(\Psi''\Psi - (\Psi')^2)/\Psi^2 = g''$):

$$\mu'(x) = -\frac{1}{x^2} - g''.$$

Progressive requires $\mu' > 0$, i.e., $g'' < -1/x^2$.

At the boundary $g'' = -1/x^2$, integrating gives $g = \ln x + cx + c_0$, so $\Psi(x) = Axe^{cx}$. For this boundary function, $\mu(x) = 1/x - (1/x + c) = -c$. Positive fees require $\mu > 0$, hence $c < 0$.

Moving past the boundary to the progressive side ($g'' < -1/x^2$) requires $c > 0$, but $c > 0$ gives $\mu = -c < 0$—negative fees. The boundary between progressive and anti-progressive coincides exactly with zero fees: no Ψ can have both positive fees and progressive structure. □

Theorem 9.10 (Ratio-only dependence requires power functions)**.** *For $\Psi(x)/\Psi(x + \Delta)$ to depend only on the reserve ratio $r = x/(x + \Delta)$ (and not on x independently), we must have $\Psi(x) = Ax^\alpha$.*

Proof. Require $\Psi(x)/\Psi(x + \Delta) = g(r)$ for some function g. Since $x + \Delta = x/r$, substituting $u = x/r$ gives $\Psi(ur)/\Psi(u) = g(r)$. Taking $h = \ln \Psi$: $h(ur) - h(u) = \ln g(r)$. Setting $u = 1$: $h(r) - h(1) = \ln g(r)$. With $H(x) := h(x) - h(1)$, this becomes $H(ur) = H(u) + H(r)$—Cauchy's multiplicative equation. The continuous solutions are $H(x) = \alpha \ln x$, giving $\Psi(x) = Ax^\alpha$. □

Remark 9.11. Ratio-only dependence ensures *scale invariance*: a 10% trade has the same fee rate regardless of pool size. This is the property that singles out power functions—not telescoping, which holds for any Ψ.

10 Economic Analysis of the Dissipative Design

We now quantify the anti-progressive structure implied by the impossibility theorem for the dissipative power-curve design.

10.1 Effective fee rate

Definition 10.1 (Effective fee rate vs constant product)**.** For a dissipative swap with reserve ratio $r = x/(x + \Delta x) \in (0, 1)$ and power $\alpha \in (0, 1)$, the fraction of constant-product output retained by the pool is:

$$\varphi_{\text{eff}}(r) = \frac{r^\alpha - r}{1 - r}. \tag{12}$$

Theorem 10.2 (Anti-progressive structure)**.** *For $0 < \alpha < 1$, φ_{eff} is strictly decreasing in trade size (i.e., increasing in r):*

- *As $r \to 1$ (small trades): $\varphi_{\text{eff}} \to 1 - \alpha$.*
- *As $r \to 0$ (large trades): $\varphi_{\text{eff}} \to 0$.*

Proof. The small-trade limit follows by L'Hôpital's rule: $\lim_{r\to 1}(r^\alpha - r)/(1-r) = \lim_{r\to 1}(\alpha r^{\alpha-1} - 1)/(-1) = 1 - \alpha$. The large-trade limit is immediate: $(0-0)/1 = 0$. For the intermediate range, expand $r^\alpha = r \cdot e^{(\alpha-1)\ln r} \approx r(1 + (\alpha - 1)\ln r)$ for α near 1, giving $\varphi_{\text{eff}} \approx (1-\alpha) \cdot r|\ln r|/(1-r)$, which decreases as r decreases (trade size increases) because the r factor dominates $|\ln r|$. □

10.2 Fee rate by trade size ($\alpha = 0.997$)

For $\alpha = 0.997$ (targeting ∼0.3% fee on small trades) with pool reserves $x = y = 1000$:

Trade Size	Δx	r	Δy_{cp}	**Fee Rate**	**vs 1%**
1% of pool	10	0.9901	9.901	0.299%	—
5% of pool	50	0.9524	47.619	0.293%	−2.0%
10% of pool	100	0.9091	90.909	0.286%	−4.3%
20% of pool	200	0.8333	166.667	0.271%	−9.4%
50% of pool	500	0.6667	333.333	0.243%	−18.7%

The "vs 1%" column shows the percentage reduction in fee rate compared to the smallest trade. Typical arbitrage trades (10–20% of pool) receive a ∼5–10% fee discount; this is modest but systematic.

10.3 Comparison with alternative fee mechanisms

Trade Size	**Flat Fee**	**Dissipative**	**Progressive***	**Desired Direction**
1%	0.30%	0.30%	0.30%	Low
10%	0.30%	0.29%	0.35%	Medium
20%	0.30%	0.27%	0.45%	Higher
50%	0.30%	0.24%	0.80%	High

**Progressive fees are illustrative (e.g., cumulative volume approach).*

10.4 Alternative: cumulative volume design

The impossibility theorem (Theorem 9.6) shows that achieving progressive + LP-protective requires state. The simplest such design tracks *cumulative volume* V per pool:

$$f(\Delta; V) = F(V + \Delta) - F(V),$$

where F is convex (e.g., $F(V) = \kappa V^2$). The marginal fee rate $F'(V)$ increases with V, giving progressivity. Telescoping gives LP-protection: splits sum to the same total by construction.

Cost: One storage slot per pool (or per order, in the SwapVM setting).

Benefit: Progressive + LP-protective.

10.5 Why stateless is preferred for the dissipative design

Despite the alternative, we adopt the stateless dissipative design for the following reasons:

1. **Gas cost.** Each storage write (SSTORE) costs ∼20,000 gas on initial write, ∼5,000 on update. The dissipative design requires *zero* additional storage—the output is computed purely from reserves and α.

2. **SwapVM compatibility.** In the SwapVM architecture, pricing instructions execute within a bytecode VM. Adding per-order or per-pool storage for cumulative volume would require new opcodes, storage management, and garbage collection logic. The stateless design fits naturally as a pure-function instruction.

3. **Simplicity and auditability.** The entire fee mechanism is a single formula: $y' = y(x/(x + \Delta x))^{\alpha}$. There is no state to initialize, decay, or migrate.

4. **Modest practical impact.** Within the realistic trade range (1–20% of pool), the anti-progressive discount is $\leq 10\%$. For most maker strategies, this is an acceptable trade-off against the benefits above.

Remark 10.3 (When to prefer cumulative volume). If a pool consistently faces large arbitrage trades ($> 20\%$ of reserves) and LP protection is the top priority, the cumulative volume approach may be preferable despite the storage cost. The choice is a design parameter, not a correctness issue.

11 Numeric showcase and evidence

We provide concrete numeric examples (with fully expanded calculations), a fee-rate visualization, and a round-trip loss summary.

11.1 Fee rate vs trade size ($\alpha = 0.997$)

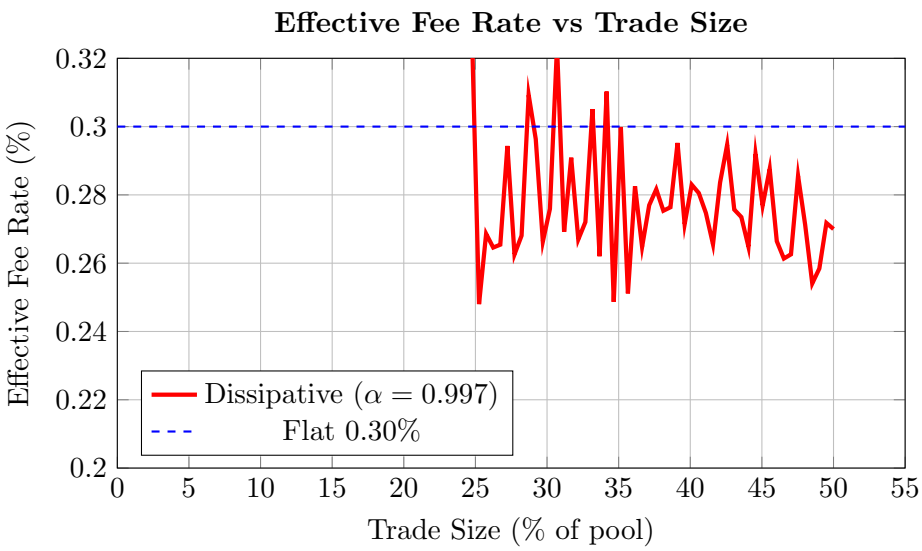

Figure 2: The effective fee rate decreases as trade size increases (anti-progressive). For $\alpha = 0.997$, the fee drops from $\sim$0.30% for small trades to $\sim$0.24% at 50% of pool. The dashed line shows a flat 0.30% fee for comparison.

11.2 Round-trip loss summary ($\alpha = 0.997$)

The dissipative design collects fees through round-trip dissipation: a trader who buys Y and immediately sells it back receives less X than they started with.

Trade Size	Trader Receives Back	Round-Trip Loss	Annualized Fee
$\Delta x = 10$ (1%)	≈ 9.994	$\approx 0.06\%$	$\sim 0.6\%$
$\Delta x = 100$ (10%)	≈ 99.43	$\approx 0.57\%$	$\sim 0.57\%$
$\Delta x = 200$ (20%)	≈ 198.9	$\approx 0.55\%$	$\sim 0.55\%$

The round-trip loss is approximately $2(1-\alpha) \approx 0.6\%$ for small trades (twice the one-way fee), decreasing slightly for larger trades due to the anti-progressive structure.

11.3 Detailed hand-checkable examples

Example 1 (not strict additive): Let $x = y = 1000$, $\alpha = 0.8$, and swap $\Delta x_{\text{in}} = 100$ in the $X \to Y$ direction.

$$\begin{aligned}
\text{(CP baseline)} \quad y' &= \frac{xy}{x + \Delta x_{\text{in}}} = \frac{1000 \cdot 1000}{1100} = \frac{1{,}000{,}000}{1100} \approx 909.0909090909, \\
\Delta y_{\text{out}}^{\text{CP}} &= 1000 - y' \approx 90.9090909091.
\end{aligned}$$

For the power rule (Sections 6–7):

$$\begin{aligned}
r &= \frac{x}{x + \Delta x_{\text{in}}} = \frac{1000}{1100} = \frac{10}{11} \approx 0.9090909090909, \\
\ln r &\approx -0.0953101798043, \\
r^{0.8} &= \exp(0.8 \ln r) = \exp\big(-0.0762481438435\big) \approx 0.9265862513559, \\
y_1 &= 1000\, r^{0.8} \approx 926.5862513558695, \\
\Delta y_{\text{out}} &= 1000 - y_1 \approx 73.4137486441305.
\end{aligned}$$

If we now swap back $Y \to X$ by inputting $\Delta y_{\text{in}} = \Delta y_{\text{out}}$ so that $y_2 = 1000$, the update gives

$$\begin{aligned}
\left(\frac{y_1}{1000}\right)^{0.8} &= \exp\Big(0.8 \ln\big(\tfrac{y_1}{1000}\big)\Big) = \exp\big(-0.0609985150748\big) \approx 0.9408246368292, \\
x_2 &= 1100 \left(\frac{y_1}{1000}\right)^{0.8} \approx 1034.9071005121361, \\
\Delta x_{\text{back}} &= 1100 - x_2 \approx 65.0928994878639.
\end{aligned}$$

Here $\Delta x_{\text{back}} < 100$, so doing the round trip loses value (not strict additive).

Example 2 (close to strict additive): Same setup, but $\alpha = 0.997$.

$$\begin{aligned}
r^{0.997} &= \exp(0.997 \ln r) = \exp\big(-0.0950242492649\big) \approx 0.9093508831104, \\
y_1 &= 1000\, r^{0.997} \approx 909.3508831104054, \\
\Delta y_{\text{out}} &= 1000 - y_1 \approx 90.6491168895946.
\end{aligned}$$

Swap back $Y \to X$ with $\Delta y_{\text{in}} = \Delta y_{\text{out}}$:

$$\begin{aligned}
\left(\frac{y_1}{1000}\right)^{0.997} &= \exp\Big(0.997 \ln\big(\tfrac{y_1}{1000}\big)\Big) = \exp\big(-0.0947391765171\big) \approx 0.9096101512187, \\
x_2 &= 1100 \left(\frac{y_1}{1000}\right)^{0.997} \approx 1000.5711663406179, \\
\Delta x_{\text{back}} &= 1100 - x_2 \approx 99.4288336593821.
\end{aligned}$$

Now Δx_{back} is much closer to 100, i.e. the round-trip loss is small.

Example 3 (ExactOut input computation): Let $x = y = 1000$, $\alpha = 0.8$, and request an *ExactOut* swap $X \to Y$ with $\Delta y_{\text{out}} = 50$. From Sections 6–7,

$$\Delta x_{\text{in}} = x\left(\left(\frac{y}{y - \Delta y_{\text{out}}}\right)^{1/\alpha} - 1\right) = 1000\left(\left(\frac{1000}{950}\right)^{1.25} - 1\right).$$

Expanding the power:

$$\begin{aligned}
\frac{1000}{950} &= \frac{20}{19} \approx 1.0526315789474,\\
\ln\left(\tfrac{20}{19}\right) &\approx 0.0512932943876,\\
\left(\tfrac{20}{19}\right)^{1.25} &= \exp\Big(1.25\ln\left(\tfrac{20}{19}\right)\Big) = \exp\big(0.0641166179844\big) \approx 1.0662167315579,\\
\Delta x_{\text{in}} &= 1000\big(1.0662167315579 - 1\big) \approx 66.2167315578528.
\end{aligned}$$

12 Pitfall: scaling ExactOut input by a constant breaks strict additivity

Suppose you compute conservative power ExactOut base input

$$\Delta x_{\text{base}} = x\left(\left(\frac{y}{y - \Delta y}\right)^{1/\alpha} - 1\right)$$

and then charge

$$\Delta x_{\text{new}} = c\,\Delta x_{\text{base}}, \quad c > 1.$$

This generally breaks ExactOut strict additivity in Δy because the correct telescoping structure requires endpoint multiplicativity in the ratio $y/(y - \Delta y)$, not post-hoc scaling of Δx. To preserve strict additivity, one must modify the exponent (an endpoint function), not multiply the solved Δx.

13 Summary

- Strict additivity for ExactIn with reinvested-within-pricing factors forces a cocycle identity and therefore a telescoping endpoint representation via a potential Ψ (Sections 3–4).
- **Conservative design**: a single potential function Ψ yields invariant $K = y\Psi(x)$, strict additivity for ExactIn/ExactOut, and invertibility (no round-trip extraction). Time-reversible (Section 5).
- **Dissipative design**: a single deterministic mapping where the power α is always applied to the input token's reserve. Strictly additive per direction, but non-conservative—round trips dissipate trader value into the pool, creating a real bid-ask spread. Time-irreversible (Section 7).
- **Standard fee rules break strict additivity**: the Curve-style output-fee rule is *superadditive* (split better for trader) because the retained fee pushes k upward, enriching subsequent chunks. The input-fee reinvest rule is *subadditive* (one-shot better) because the extra credited input makes subsequent same-direction trades less favorable. Both are path-dependent (Section 8).
- **Strict additivity matters** for aggregator/router neutrality, partial fills, deterministic pool growth, composability with the SwapVM architecture, and eliminating gaming incentives from split strategies (Section 8).

- **Impossibility theorem**: no stateless fee function can simultaneously be progressive and LP-protective. The proof uses infinitesimal splitting: state changes are second-order while progressive rate reduction is first-order, so fine-grained splitting always reduces total fees—contradicting LP-protection (Section 9).
- **Economic consequence**: the dissipative design is necessarily *anti-progressive* ($\sim$10% fee discount for typical arbitrage trades at 10–20% of pool). This is modest and acceptable given the benefits of statelessness, gas efficiency, and SwapVM compatibility (Section 10).
- **Alternative**: cumulative volume fees achieve progressive + LP-protective at the cost of one storage slot per pool. The choice between stateless-dissipative and stateful-progressive is a design parameter, not a correctness issue.

13.1 Mechanism comparison

Mechanism	Additivity	Stateless?	Progr.?	Best For
Fee-free $xy = k$	Strict	✓	N/A	(baseline)
Output-fee (Curve)	Superadditive	✓	×	Legacy
Input-fee reinvest	Subadditive	✓	×	Legacy
Dissipative r^α	**Strict**	✓	× (anti)	**Recommended**
Flat fees	Strict	✓	×	Simple pools
Cumulative volume	Strict	×	✓	When progressive needed

13.2 Decision table

If you need...	Use...	Accept...
Stateless + LP-Protective	Dissipative r^α	Anti-progressive ($\sim$ 10% whale discount)
Progressive + LP-Protective	Cumulative volume	1 state variable per pool
Stateless + Progressive	Per-trade progressive	Exploitable by splitting

References

- Uniswap v2 Whitepaper: https://app.uniswap.org/whitepaper.pdf
- Angeris et al., Constant Function Market Makers (CFMM): https://web.stanford.edu/~boyd/papers/pdf/cfmm.pdf
- CFMM geometry and axioms (optional reading): https://arxiv.org/pdf/2308.08066.pdf, https://arxiv.org/pdf/2210.00048.pdf, https://arxiv.org/pdf/2302.00196.pdf